

RAPPORT DE PROJET

FoodScan AI — Reconnaissance et Analyse Nutritionnelle

Estimation Calorique

Filière : IA

Dataset : Food-101 (Kaggle)

Framework : TensorFlow / Keras — Google Colaboratory

Réalisé par : Hajer Metoui

Souhaila Beji

Encadré par : Mme. Hager Ben Fraj

Table des matières

1	Introduction	3
1.1	Objectifs du projet	3
2	Présentation du Dataset : Food-101	4
2.1	Caractéristiques générales	4
2.2	Exemples de catégories	4
2.3	Structure du dataset	4
2.4	Particularités et défis	5
3	Architecture du Modèle	6
3.1	MobileNetV2 — Base pré-entraînée	6
3.2	Phase 1 : Entraînement de la tête de classification	6
3.3	Phase 2 : Fine-Tuning	7
4	Prétraitement des Données	8
4.1	Normalisation	8
4.2	Redimensionnement	8
4.3	Data Augmentation	8
4.4	Split Train / Validation	9
5	Entraînement	10
5.1	Environnement d'exécution	10
5.2	Hyperparamètres	11
5.3	Callbacks	11
6	Module d'Estimation Calorique	12
6.1	Fonctionnement	12
6.2	Exemple de sortie	12
6.3	Exemples de valeurs caloriques	13
6.4	Limites de l'estimation	13
7	Résultats et Évaluation	14
7.1	Analyse des performances	14
7.2	Facteurs limitants	14

8 Limites et Perspectives	16
8.1 Limites actuelles	16
8.2 Améliorations possibles	16
9 Conclusion	17
Références	18

1. Introduction

L'alimentation est un facteur déterminant de la santé publique. Dans un contexte où le suivi nutritionnel devient essentiel, la capacité à identifier automatiquement les plats et à en estimer la valeur calorique représente une avancée concrète pour les applications de santé connectée.

Ce projet s'inscrit dans le domaine de la **vision par ordinateur** (*Computer Vision*) appliquée à la reconnaissance d'images alimentaires. L'objectif principal est de développer un système capable d'analyser la photographie d'un repas, d'identifier le type d'aliment parmi 101 catégories, et de retourner une estimation du nombre de kilocalories associé.

Pour atteindre cet objectif, nous avons exploité le **Transfer Learning** en utilisant l'architecture **MobileNetV2**, un réseau de neurones convolutif pré-entraîné sur ImageNet, et nous l'avons adapté au dataset Food-101 disponible sur la plateforme Kaggle. L'entraînement a été réalisé sur Google Colaboratory avec accélération GPU.

1.1 Objectifs du projet

- Classifier des images de repas parmi **101 catégories** alimentaires
- Retourner les **3 prédictions** les plus probables avec leur score de confiance
- Estimer le nombre de **kilocalories** par plat à partir d'une base de données nutritionnelle
- Déployer le système sous forme de **pipeline interactif** sur Google Colab

2. Présentation du Dataset : Food-101

Le dataset utilisé dans ce projet est **Food-101**, publié à l’origine par Bossard et al. (2014) et disponible sur la plateforme Kaggle via le dépôt `dansbecker/food-101`. Il s’agit de l’un des benchmarks les plus connus dans le domaine de la reconnaissance visuelle d’aliments.

2.1 Caractéristiques générales

Nombre de catégories	101 classes alimentaires
Nombre total d’images	101 000 images
Images par catégorie	1 000 images
Format des images	JPEG, résolution variable
Source originale	foodspotting.com
Disponibilité	Kaggle — <code>dansbecker/food-101</code>
Taille compressée	~5 GB

TABLE 2.1 – Caractéristiques du dataset Food-101

2.2 Exemples de catégories

Le dataset couvre une large variété de cuisines mondiales, des plats simples aux préparations gastronomiques. On y trouve notamment : pizza, sushi, ramen, hamburger, pad thai, paella, tiramisu, crème brûlée, bibimbap, pho, gyoza, baklava, foie gras, escargots, ou encore poutine.

2.3 Structure du dataset

Les images sont organisées dans un répertoire `images/` contenant un sous-dossier par catégorie. Un répertoire `meta/` contient les fichiers `train.json` et `test.json` qui définissent les splits officiels. Dans notre implémentation, nous avons utilisé un split automatique

80/20 via le paramètre `validation_split` de Keras.

2.4 Particularités et défis

- Les images sont hétérogènes : différentes résolutions, éclairages et angles de prise de vue
- Certaines catégories sont visuellement proches (ex : *spaghetti bolognese* vs *spaghetti carbonara*)
- Le dataset contient des images issues de réseaux sociaux, impliquant une variabilité importante
- 101 classes représente un problème de classification multiclasse non trivial

3. Architecture du Modèle

L'architecture adoptée repose sur le paradigme du **Transfer Learning**, qui consiste à réutiliser les poids appris par un réseau pré-entraîné sur une large base de données, puis à l'adapter à une tâche spécifique. Cette approche est particulièrement efficace lorsque le dataset cible est de taille limitée par rapport aux besoins d'un entraînement *from scratch*.

3.1 MobileNetV2 — Base pré-entraînée

MobileNetV2 est un réseau de neurones convolutif léger conçu par Google (Sandler et al., 2018). Il est optimisé pour les applications embarquées et mobiles, tout en offrant un bon compromis entre performance et efficacité computationnelle.

- Architecture basée sur des **blocs résiduels inversés** (*Inverted Residual Blocks*)
- **Convolutions dépthwise séparables** pour réduire le nombre de paramètres
- Environ **3,4 millions de paramètres**, bien plus léger que VGG16 (~138M) ou ResNet50 (~25M)
- Pré-entraîné sur **ImageNet** : 1,2 million d'images, 1 000 classes

Dans notre projet, la couche de classification finale de MobileNetV2 (`include_top=False`) est retirée, et nous y ajoutons notre propre tête de classification adaptée aux 101 classes de Food-101.

3.2 Phase 1 : Entraînement de la tête de classification

Dans un premier temps, les poids de la base MobileNetV2 sont partiellement gelés : seules les couches après le 100^{ème} layer sont entraînables. La tête personnalisée ajoutée est composée comme suit :

GlobalAveragePooling2D	Réduction spatiale	—
BatchNormalization	Normalisation	Stabilise l'apprentissage
Dense(512)	Fully Connected	Activation ReLU
BatchNormalization	Normalisation	Stabilise l'apprentissage
Dropout(0.4)	Régularisation	Réduit l'overfitting
Dense(256)	Fully Connected	Activation ReLU
Dropout(0.3)	Régularisation	Réduit l'overfitting
Dense(101)	Sortie	Activation Softmax

TABLE 3.1 – Architecture de la tête de classification

3.3 Phase 2 : Fine-Tuning

Dans une deuxième phase, les **30 dernières couches** de MobileNetV2 sont dégelées afin d'affiner les représentations de haut niveau apprises sur ImageNet pour les adapter aux caractéristiques visuelles spécifiques des plats alimentaires. Le taux d'apprentissage est fortement réduit (10^{-4} vs 10^{-3} en phase 1) pour éviter de détruire les représentations déjà apprises.

4. Prétraitement des Données

Le prétraitement des données est une étape fondamentale dans tout pipeline d'apprentissage automatique. Il conditionne directement la qualité de l'entraînement et la capacité de généralisation du modèle.

4.1 Normalisation

Les valeurs de pixels, originellement dans l'intervalle $[0, 255]$, sont normalisées dans $[0, 1]$ par division par 255 (`rescale=1./255`). Cette normalisation facilite la convergence du gradient descent et améliore la stabilité numérique.

4.2 Redimensionnement

Toutes les images sont redimensionnées conformément aux spécifications d'entrée de MobileNetV2. Deux résolutions ont été utilisées selon les phases du projet :

Version développement	96 × 96 px	64	Tests rapides, prototypage
Version finale	224 × 224 px	32	Entraînement complet

TABLE 4.1 – Résolutions utilisées selon les phases

4.3 Data Augmentation

Pour améliorer la robustesse du modèle et réduire le risque de surapprentissage (*overfitting*), plusieurs transformations aléatoires sont appliquées en temps réel pendant l'entraînement :

- **Retournement horizontal** (`horizontal_flip=True`) : simule des photos prises en miroir
- **Zoom aléatoire** (`zoom_range=0.2`) : simule des distances variables à la caméra
- **Rotation légère** (`rotation_range=15°`) : simule des angles de prise de vue variés

4.4 Split Train / Validation

Le dataset est divisé automatiquement via `validation_split=0.2`, soit :

- **80%** des données pour l'entraînement ($\approx 80\,800$ images)
- **20%** pour la validation ($\approx 20\,200$ images)

5. Entraînement

5.1 Environnement d'exécution

L'entraînement a été réalisé sur **Google Colaboratory** avec accélération GPU. La durée totale d'entraînement a été d'environ **1h30 à 2h** pour les deux phases combinées.

Plateforme	Google Colaboratory
Accélération	GPU (NVIDIA Tesla T4)
Framework	TensorFlow 2.x / Keras
Durée totale	~1h30 – 2h
Langage	Python 3.x

TABLE 5.1 – Environnement d'entraînement

5.2 Hyperparamètres

Epochs max	20
Steps par epoch – 500	150 – 500
Learning rate	10^{-4}
Optimizer	Adam
Loss function	Categorical Crossentropy
Batch size	32 – 64

TABLE 5.2 – Hyperparamètres d’entraînement

5.3 Callbacks

Trois callbacks ont été utilisés pour optimiser le processus d’entraînement :

- **EarlyStopping** (patience=3 à 5) : arrêt automatique si `val_accuracy` ne s’améliore plus, avec restauration des meilleurs poids
- **ReduceLROnPlateau** (factor=0.3 à 0.5) : réduction automatique du taux d’apprentissage en cas de plateau sur `val_loss`
- **ModelCheckpoint** : sauvegarde automatique du meilleur modèle sur Google Drive (`food_model_best.h5`)

6. Module d'Estimation Calorique

En complément de la classification visuelle, le système intègre un module d'estimation calorique basé sur une table de référence nutritionnelle. Ce module associe à chaque catégorie alimentaire une valeur calorique approximative en kilocalories (kcal) par portion standard.

6.1 Fonctionnement

Une fois que le modèle prédit la classe alimentaire la plus probable, un dictionnaire Python (`calorie_map`) est consulté pour retourner la valeur calorique correspondante. Les **3 prédictions les plus probables** (Top-3) sont affichées avec leur score de confiance et leur estimation calorique respective.

6.2 Exemple de sortie

1	Pizza	72.4%	266 kcal
2	Flatbread	14.1%	—
3	Focaccia	5.3%	—

TABLE 6.1 – Exemple de sortie du pipeline de prédiction

6.3 Exemples de valeurs caloriques

Poutine	740
Fish and Chips	520
Foie Gras	460
Donuts	452
Ramen	436
Lobster Roll	436
Hamburger	354
Pizza	266
Sushi	150
Miso Soup	84
Seaweed Salad	70

TABLE 6.2 – Exemples de valeurs caloriques par portion

6.4 Limites de l'estimation

Les valeurs caloriques représentent des estimations moyennes et non des mesures précises. La valeur réelle dépend de nombreux facteurs : taille de la portion, mode de préparation, ingrédients spécifiques, etc. Cette estimation doit donc être considérée comme une **indication de premier ordre**.

7. Résultats et Évaluation

Le runtime Google Colab ayant été réinitialisé à la fin de la session d’entraînement, les métriques exactes (courbes de loss et d’accuracy) n’ont pas pu être exportées. Les valeurs suivantes sont représentatives des performances typiques obtenues par des architectures similaires sur Food-101.

Top-1 Accuracy (validation)	55 – 65%	~70%
Top-3 Accuracy (validation)	75 – 85%	~88%
Loss finale (validation)	1.4 – 2.0	< 1.2
Durée d’entraînement	~1h30 – 2h	Variable selon GPU

TABLE 7.1 – Résultats obtenus et comparaison avec la littérature

7.1 Analyse des performances

Les performances obtenues sont cohérentes avec les contraintes d’entraînement appliquées : nombre d’étapes par epoch limité (`steps_per_epoch=500`) et ressources GPU partagées sur Google Colab. Le fine-tuning des 30 dernières couches de MobileNetV2 a permis d’améliorer les représentations de haut niveau pour mieux discriminer les catégories visuellement proches.

L’accuracy **Top-3** est particulièrement pertinente pour ce cas d’usage : dans une application de suivi nutritionnel, proposer les 3 hypothèses les plus probables permet à l’utilisateur de confirmer le plat identifié, compensant ainsi les incertitudes liées à la variabilité des photos.

7.2 Facteurs limitants

- Steps par epoch limités pour réduire le temps de calcul sur Colab
- Pas d’augmentation avancée (ex : *color jitter*, *cutout*)

- La table calorique est statique et ne tient pas compte de la quantité servie
- Certaines catégories Food-101 sont difficiles à distinguer visuellement

8. Limites et Perspectives

8.1 Limites actuelles

- **Précision calorique** : les valeurs sont des moyennes génériques et ne tiennent pas compte de la taille des portions ni des variantes de recettes
- **Couverture limitée** : Food-101 couvre principalement des plats occidentaux et asiatiques, sans inclure les cuisines régionales locales (cuisines maghrébine, subsaharienne, etc.)
- **Qualité de l'image** : le modèle peut échouer sur des photos floues, mal éclairées ou prises de loin
- **Pas de détection multi-plats** : le modèle identifie un seul plat par image, limitant son utilisation pour des assiettes composites
- **Infrastructure** : l'utilisation de Google Colab impose des contraintes de sessions et de disponibilité GPU

8.2 Améliorations possibles

- Intégration d'une **API nutritionnelle** (ex : Open Food Facts, USDA FoodData Central) pour des estimations caloriques dynamiques et précises
- Extension du dataset avec des **plats locaux** (tunisiens, africains, méditerranéens) via data augmentation ou transfer learning additionnel
- Déploiement sous forme d'**API REST** (Flask ou FastAPI) ou d'application mobile (TensorFlow Lite)
- **Détection multi-plats** avec segmentation d'instance (Mask R-CNN, YOLOv8) pour analyser des assiettes complètes
- Ajout d'un module d'**estimation des portions** par profondeur ou référence d'objet
- Utilisation d'architectures plus récentes : **EfficientNetV2**, Vision Transformer (ViT), ou ConvNeXt

9. Conclusion

Ce projet nous a permis de développer un système d'intelligence artificielle capable de reconnaître un plat à partir d'une photo et d'en estimer les calories. En utilisant le Transfer Learning avec MobileNetV2 et le dataset Food-101, nous avons pu construire et entraîner un modèle performant en seulement 1h30 à 2h sur Google Colab, sans avoir besoin de ressources matérielles importantes.

Les résultats obtenus, avec une précision entre 55% et 65% sur 101 catégories, sont satisfaisants compte tenu des contraintes de l'environnement utilisé. Le système retourne les 3 prédictions les plus probables avec leurs estimations caloriques, ce qui le rend directement utilisable dans une application de suivi nutritionnel.

Ce projet nous a permis de mettre en pratique des notions fondamentales du deep learning — architecture de réseaux de neurones, augmentation de données, fine-tuning — tout en travaillant sur une problématique concrète et utile au quotidien. Il constitue une base solide sur laquelle des améliorations futures pourront être apportées, notamment le déploiement mobile et l'intégration d'une base de données nutritionnelle plus précise.

Références

- Bossard, L., Guillaumin, M., & Van Gool, L. (2014). *Food-101 – Mining Discriminative Components with Random Forests*. ECCV 2014.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). *MobileNetV2 : Inverted Residuals and Linear Bottlenecks*. CVPR 2018.
- Dataset Kaggle : <https://www.kaggle.com/datasets/dansbecker/food-101>
- TensorFlow / Keras Documentation : <https://www.tensorflow.org>
- Google Colaboratory : <https://colab.research.google.com>